



山东大学

信息科学与工程学院

2024—2025 学年第二学期

# 实验报告

课程名称: 数据结构与算法导论

实验名称: 实验 3: 层次化遍历二叉树

专 业 班 级 通信工程 三班

学 生 学 号 \_\_\_\_\_

学 生 姓 名 \_\_\_\_\_

实 验 时 间 2025 年 5 月 19 日

# 实验报告

## 【实验目的】

掌握指针变量、动态变量的含义，掌握二叉树的结构特征，以及各种存储结构的特点及适用范围，掌握指针类型描述、访问和处理二叉树的运算；

## 【实验原理】

本次实验，我们需要构建一棵二叉树，根据课上所学，我们二叉树应该使用递归方式来构建结构为：

```
typedef struct TreeNode {  
    int data;  
    struct TreeNode* left;  
    struct TreeNode* right;  
} TreeNode;
```

我们需要对构建出的二叉树进行层次化遍历，即按以下步骤进行迭代访问

- (1) 访问根节点
- (2) 访问左子树根节点
- (3) 访问右子树根节点

由于需要对于数据先入先出，因此使用队列是必要的，那么结构为

- (1) 根节点入队列退队列输出
- (2) 左子树根节点入队列
- (3) 右子树根节点入队列

## 【实验要求】

实现二叉树表的层次遍历算法，并给出应用。

## 【实验源程序】

### 1.bitree 头文件

```
#ifndef _bitree_H
#define _bitree_H

#define Maxsize 255
//定义树节点
typedef struct TreeNode {
    int data;
    struct TreeNode* left;
    struct TreeNode* right;
} TreeNode;

TreeNode* createNode(int data); //创建树节点
//void levelOrderTraversal(TreeNode* root);
void levelOrderTraversal(TreeNode* bintree); //层次遍历

#endif
```

### 2.bitree 函数文件

```
#include<stdio.h>
#include "bitree.h"
#include "Queue.h"
#include "ADT.h"

/* 创建二叉树节点
 * 参数: data 节点存储的数据
 * 返回: 新创建的节点指针
 */
TreeNode* createNode(int data) {
    TreeNode* node = (TreeNode*)malloc(sizeof(TreeNode));
    node->data = data; // 设置节点数据
    node->left = NULL; // 初始化左子树指针
    node->right = NULL; // 初始化右子树指针
    return node;
}

/*是否空树
```

```

    * 参数:  bintree 二叉树根节点指针
    * 返回:  0 表示空, 1 表示非空
    */
int isEmpty_T(TreeNode* bintree) {
    if(bintree->data == NULL) // 通过数据字段判断空树 (错误方式)
        return 0;
    return 1;
}

/* 层次遍历二叉树 (循环队列实现)
    * 参数:  bintree 二叉树根节点指针
    * 返回:  无
    */
void levelOrderTraversal(TreeNode* bintree) {
    TreeNode* q;           // 当前处理节点指针
    TreeNode* T[Maxsize];  // 队列存储容器
    int front, rear;        // 队列指针

    // 初始化队列指针
    rear = -1;              // 空队列标记
    front = -1;

    // 空树检查
    if(isEmpty_T(bintree) == 0)
        return;

    // 根节点入队操作
    T[0] = bintree;         // 存储根节点到队列
    rear = (rear + 1) % Maxsize; // 循环队列尾指针移动

    // 层次遍历主循环
    while(front != rear) {   // 队列不为空时继续
        // 出队操作
        front = (front + 1) % Maxsize; // 循环队列头指针移动
        q = T[front];           // 获取队首元素
        printf("%d ", q->data); // 输出根节点

        // 左子树入队
        if(q->left != NULL) {
            rear = (rear + 1) % Maxsize; // 尾指针循环移动
            T[rear] = q->left; // 存储左子节点
        }

        // 右子树入队
    }
}
    
```

```

        if(q->right != NULL) {
            rear = (rear + 1) % Maxsize; // 尾指针循环移动
            T[rear] = q->right; // 存储右子节点
        }
    }
}

```

### 3.主程序应用示例

```

#include<stdio.h>
#include "ADT.h"
#include "Queue.h"
#include "bitree.h"

int main() {
    //手动创造一棵二叉树
    TreeNode* root = createNode(10);
    root->left = createNode(5);
    root->right = createNode(15);
    root->left->left = createNode(3);
    root->left->right = createNode(7);
    root->right->left = createNode(12);
    root->right->right = createNode(20);

    // 执行层次遍历
    printf("层次遍历结果: ");
    levelOrderTraversal(root);
    // 手动释放内存
    free(root->right->right);
    free(root->right->left);
    free(root->right);
    free(root->left->right);
    free(root->left->left);
    free(root->left);
    free(root);
    return 0;
}

```

## 【实验结果】

我们构建的二叉树如下图所示：

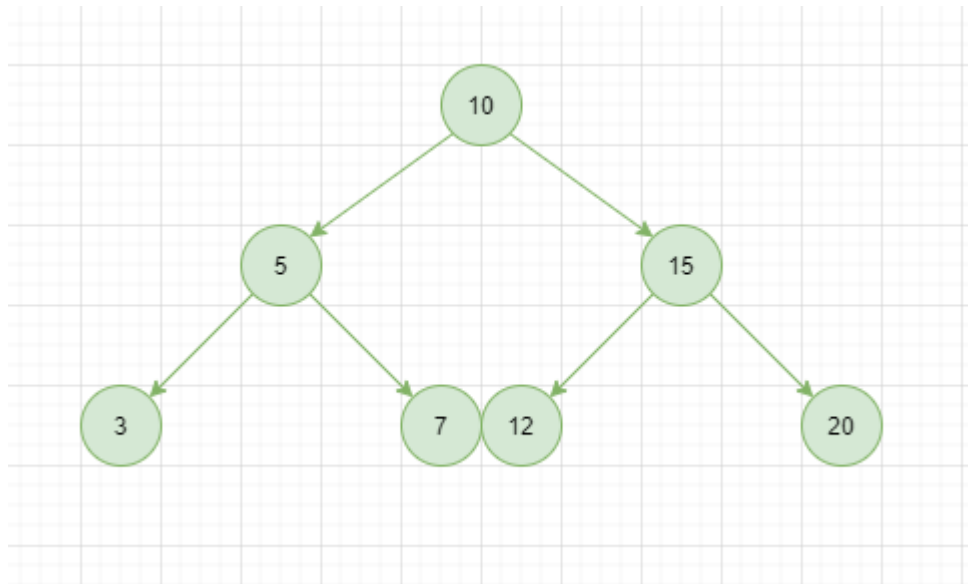


图 1 构造二叉树的结构

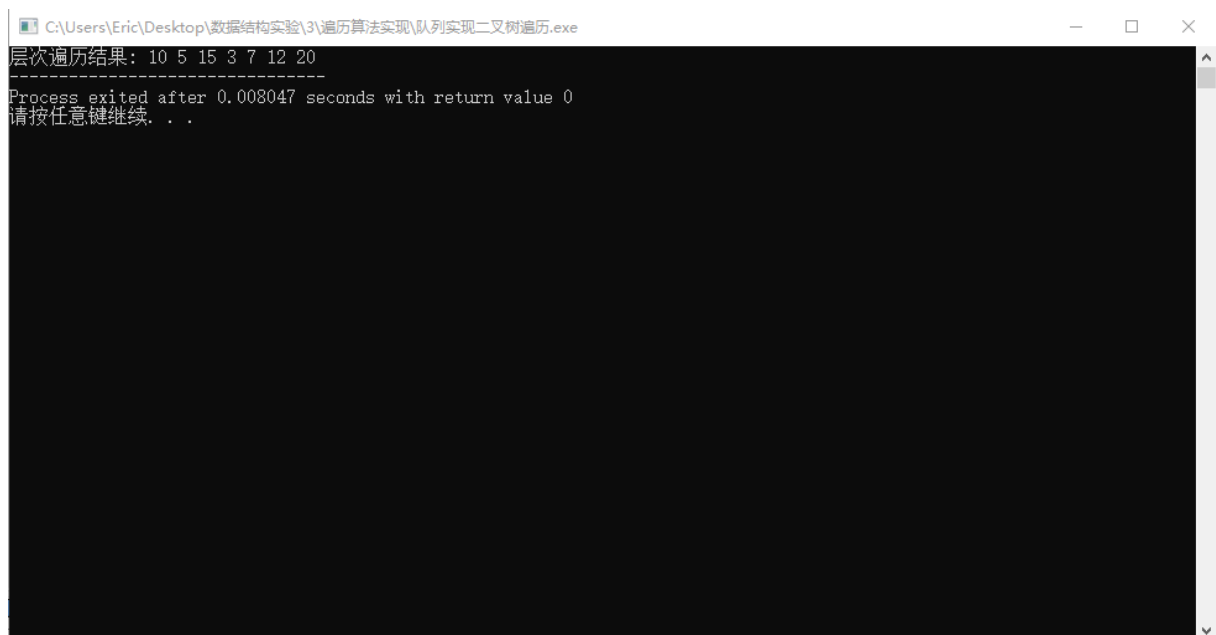


图 2 运行结果截图

本次实验中，为使二叉树构建更加直观，因此我们并未通过递归抑或是迭代来实现构造二叉树和释放二叉树的空间，而是一个个的手动操作来实现。我们构造的二叉树结构如上图，同样的，我们输出的正是每一层的节点值，因此，实验非常成功。

## 【实验心得】

本次实验通过对二叉树的构建与层次化遍历，我对二叉树的特性与队列的特性有了更深的了解，我在实验中也遇到很多困难，以下是我主要遇到的困难：

### 1.为层次化遍历构造队列

一开始，我想通过上次实验的已经构造的 ADT 队列来继续构造队列，但是我却忽略我所存的数据是 INT 类型，如果需要调用它们的话，需要重写队列文件，或者只让 data 入队列，我选择了后者，事实证明，这是一个错误的想法。

```
//void levelOrderTraversal(TreeNode* root) {  
//    Queue q;  
//    initQue(&q);  
  
//    if (root != NULL) {  
//        // 将根节点指针转换为 int 入队  
//        enQue(&q, root->data);  
//    }  
  
//    while (!isEmpty_Q(&q)) {  
//        int element;  
//        // 出队节点指针  
//        if (deQue(&q, &element)) {  
//            TreeNode* current = (TreeNode*)element;  
//  
//            // 访问节点数据  
//            printf("%d", current->data);  
  
//            // 左子树入队  
//            if (current->left != NULL) {  
//                enQue(&q, (current->left->data));  
//            }  
  
//            // 右子树入队  
//            if (current->right != NULL) {  
//                enQue(&q, (current->right->data));  
//            }  
//        }  
//    }  
//}
```

```

        // 销毁队列
    //    del_Queue(&q);
    //}
    
```

它只能输出第 1、2 层树的节点，这是因为 data 不是指针，不能指向下一个子树，我顿然醒悟，我需要让节点入队列完成迭代，于是，我选择构造一个指针数组来实现队列。

```

void levelOrderTraversal(TreeNode* bintree) {
    TreeNode* q;           // 当前处理节点指针
    TreeNode* T[Maxsize];  // 队列存储容器
    int front, rear;        // 队列指针

    // 初始化队列指针
    rear = -1;              // 空队列标记
    front = -1;

    // 空树检查
    if(isEmpty_T(bintree) == 0)
        return;

    // 根节点入队操作
    T[0] = bintree;         // 存储根节点到队列
    rear ++; // 循环队列尾指针移动

    // 层次遍历主循环
    while(front != rear) {  // 队列不为空时继续
        // 出队操作
        front ++; // 循环队列头指针移动
        q = T[front];      // 获取队首元素
        printf("%d ", q->data); // 输出根节点

        // 左子树入队
        if(q->left != NULL) {
            rear ++; // 尾指针循环移动
            T[rear] = q->left; // 存储左子节点
        }

        // 右子树入队
        if(q->right != NULL) {
            rear ++; // 尾指针循环移动
            T[rear] = q->right; // 存储右子节点
        }
    }
}
    
```

从课上了解到，数组队列的缺点是空间利用率太小，于是我们构造了一个循环队列。

```
rear = (rear + 1) % Maxsize; // 尾指针循环移动
```

这要求节点的数目不能超过 MaxSize，否则会混叠。

## 2. 释放二叉树空间

本次实验中我使用手动方式释放一个二叉树的空间，但实际项目中，这样的方式有太过麻烦之嫌，应使用递归调用的方式来实现二叉树空间的回收，于是下一个实验中我多加上一个函数：

```
// 释放树的内存
void freeTree(TreeNode* root) {
    if (root == NULL) return;
    freeTree(root->left);
    freeTree(root->right);
    free(root);
}
```

很明显，这样写比我开始的写法更加简洁。